Informe Técnico y Bitácora DUIA

Proyecto: Food Store (PostgreSQL 16+, MongoDB, SQLite)

Responsable / DBA: Ojeda Lautaro, Agustina Piriz, Benjamin Cajales, Genaro Senatore, Exequiel Castro

Unidad 4: Normalización Avanzada y Redundancia

1. Elementos implementados y Decisiones de diseño

Se refactorizó un esquema heredado desnormalizado (tabla `ventas_historicas_planas`) llevándolo a la Forma Normal de Boyce-Codd (BCNF). La decisión de diseño priorizó la eliminación de redundancias mediante la creación de tablas relacionales estrictas para clientes, productos, categorías y pedidos, asegurando la integridad referencial.

2. Pruebas de funcionamiento y Resultados La migración de datos sin pérdida se ejecutó dentro de transacciones reversibles. Se probó exitosamente el plan de reversión ejecutando un `ROLLBACK` manual antes de confirmar los cambios, demostrando que ante un error de inserción, la base de datos se protege y regresa a su estado inicial vacío. Finalmente, la migración se concretó con éxito mediante un `COMMIT`.

Unidad 5: Administración y Seguridad

1. Auditoría de Seguridad Asistida por IA

Se solicitó a la IA una revisión de seguridad del esquema de roles implementado.

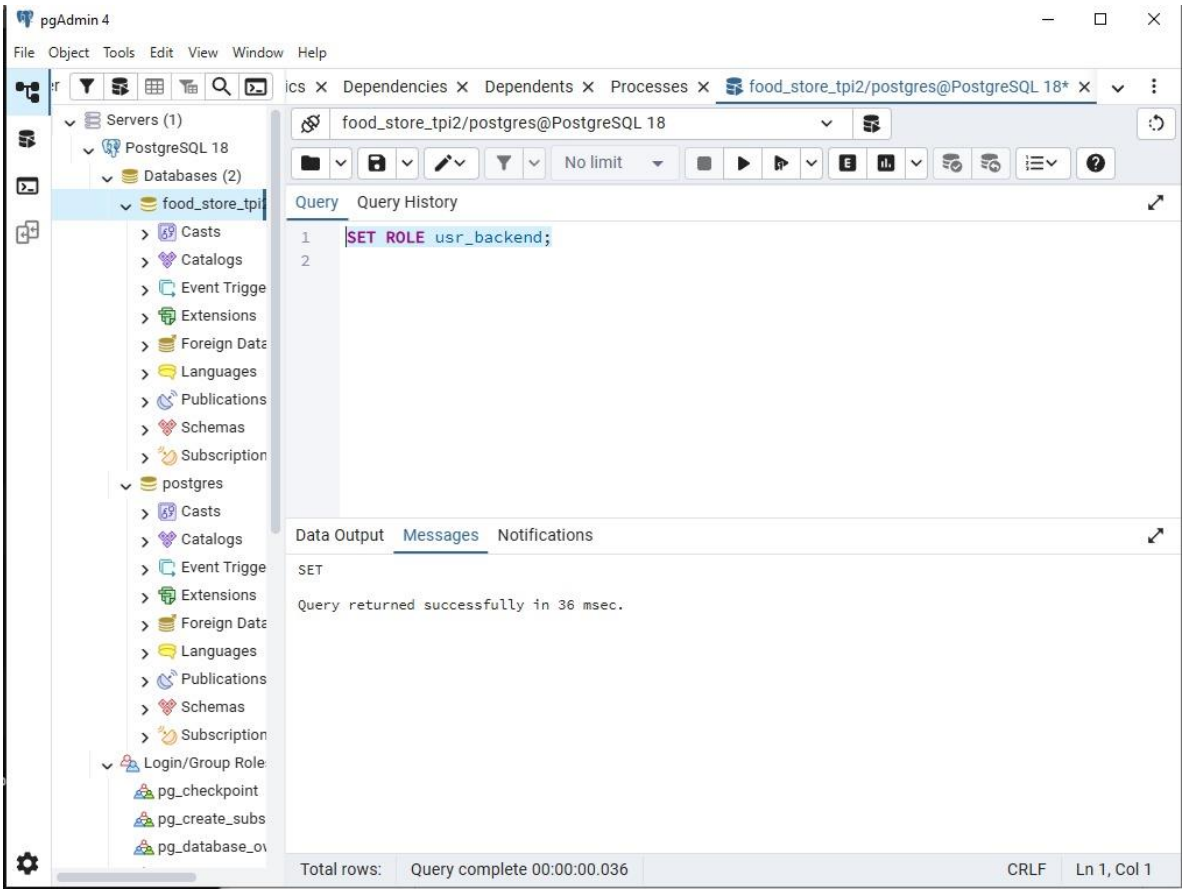
Fortalezas del Diseño Actual:

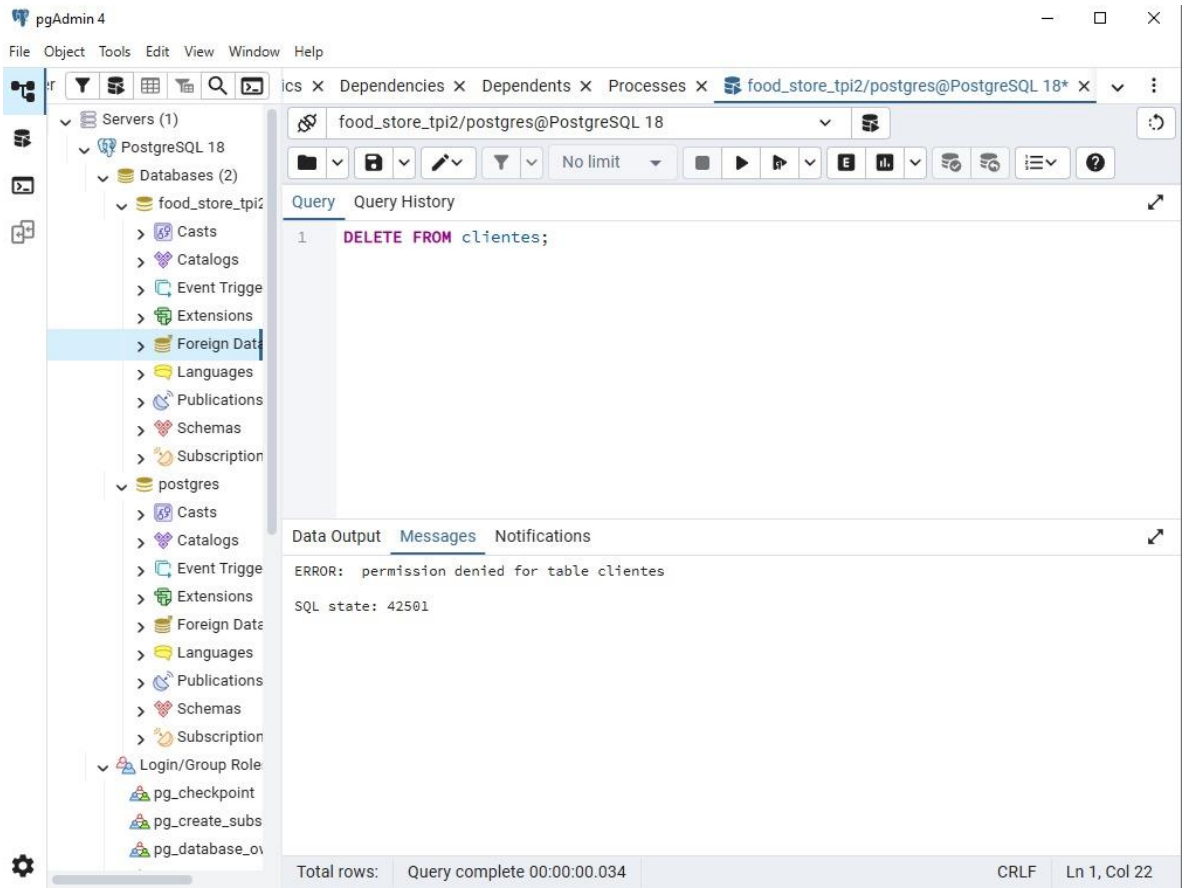
- **Principio de Mínimo Privilegio (PoLP):** El rol `rol_app_backend` está adecuadamente acotado al prohibir explícitamente el comando DELETE físico y cualquier comando DDL (DROP, ALTER), lo que mitiga el impacto de un ataque de inyección SQL destructivo o borrados accidentales masivos.
- **Trazabilidad Forense:** El uso de disparadores con `to_jsonb(OLD)` en la tabla `auditoria_operaciones` proporciona una instantánea precisa del estado anterior de los registros ante cualquier modificación o intento de borrado, cumpliendo con estándares de cumplimiento y auditoría.

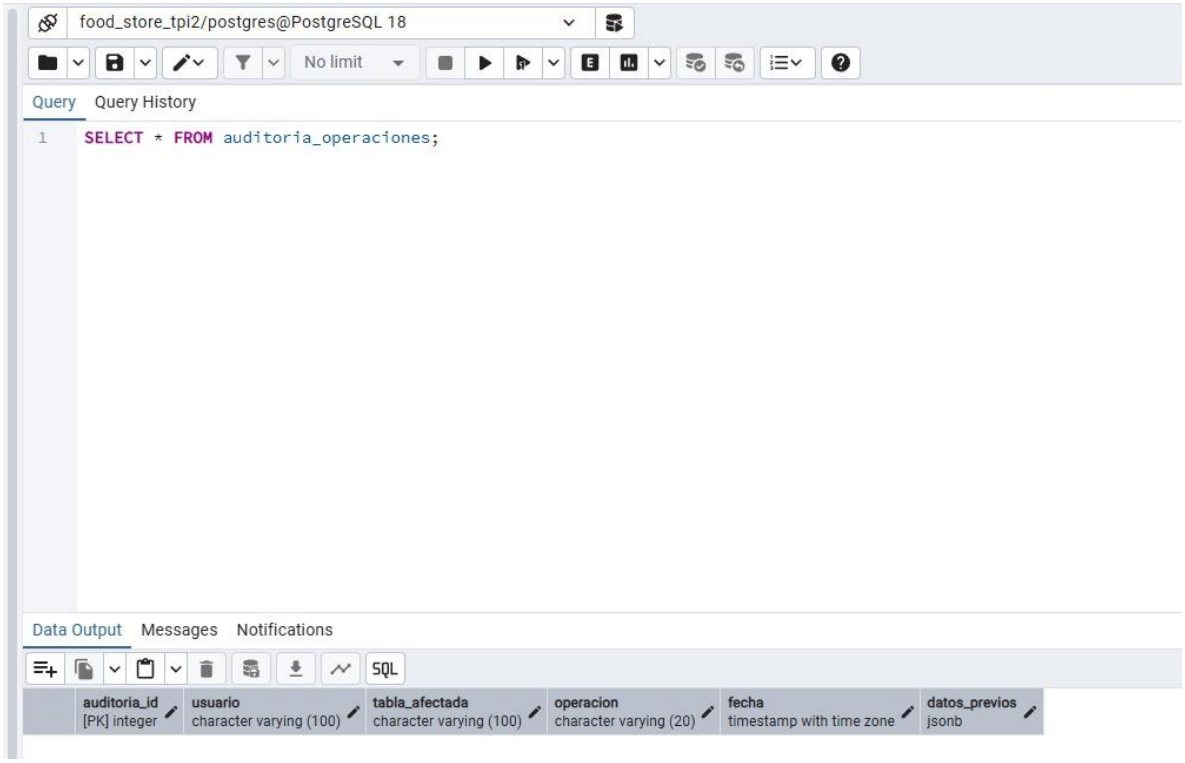
Vulnerabilidades Teóricas y Propuestas de Mejora:

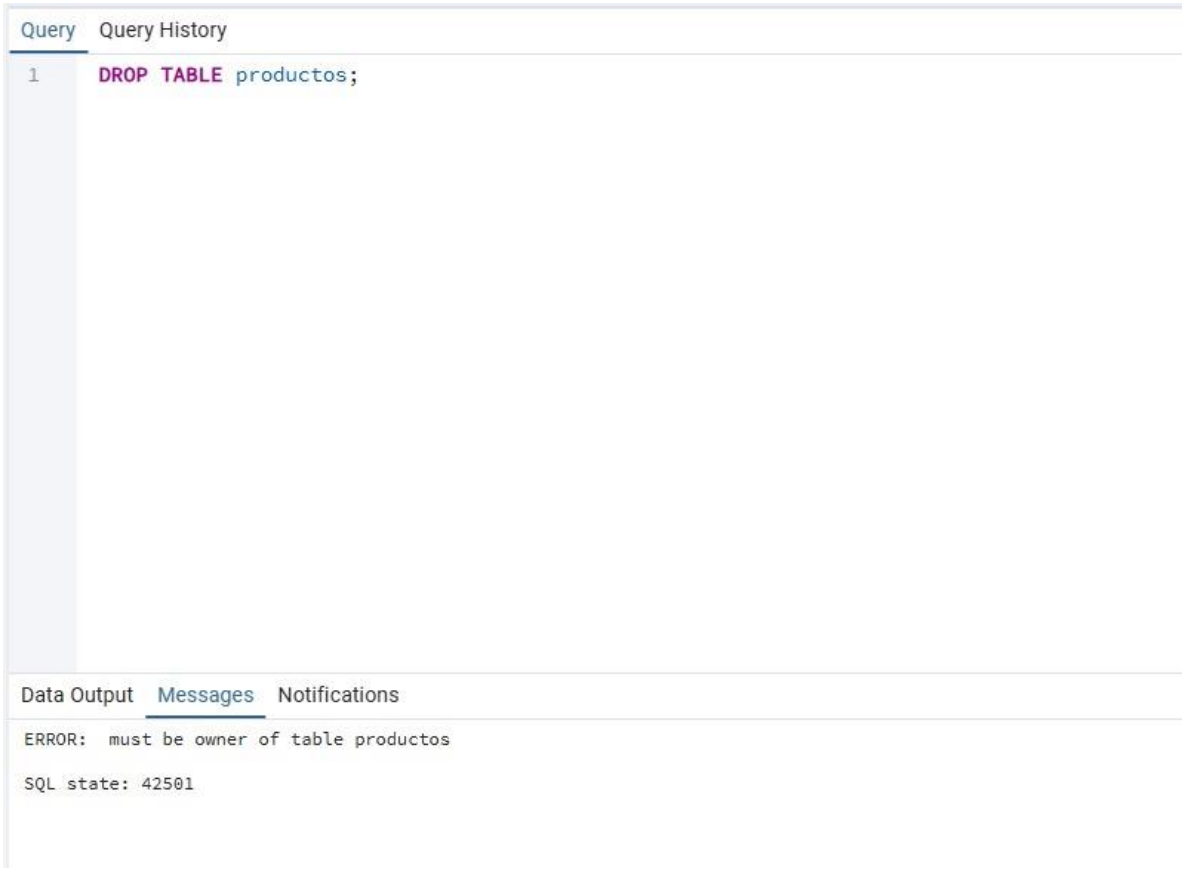
- **Vulnerabilidad:** Privilegios sobre la Tabla de Auditoría. Si un atacante logra comprometer las credenciales del `rol_admin`, podría manipular o vaciar la tabla `auditoria_operaciones`.
- **Propuesta de Mejora:** Implementar un esquema segregado (ej. schema seguridad) donde la tabla de auditoría sea de solo escritura para las funciones de trigger y solo consultable por el `rol_auditor`. Adicionalmente, implementar vistas con enmascaramiento dinámico (Data Masking) para ocultar campos sensibles (teléfonos/correos).

Decisión del Equipo: La propuesta de la IA fue aceptada; el equipo coincide en la necesidad de proteger el log, por lo que se implementará un esquema segregado de seguridad y vistas enmascaradas en la próxima versión del sistema.




pgAdmin 4
File Object Tools Edit View Window Help
Dependencies
Dependents
Processes
food_store_tpi2/postgres@PostgreSQL 18*
Servers (1)
PostgreSQL 18
Databases (2)
food_store_tpi2
Casts
Catalogs
Event Trigge
Extensions
Foreign Data
Languages
Publications
Schemas
Subscription
postgres
Login/Group Role
pg_checkpoint
pg_create_subs
pg_database_ov
food_store_tpi2/postgres@PostgreSQL 18
No limit
Query
Query History
1 DELETE FROM clientes;
Data Output
Messages
Notifications
ERROR: permission denied for table clientes
SQL state: 42501
Total rows:
Query complete 00:00:00.034
CRLF
Ln 1, Col 22


food_store_tpi2/postgres@PostgreSQL 18
No limit
Query
Query History
1 SELECT * FROM auditoria_operaciones;
Data Output
Messages
Notifications
SQL
auditoria_id
[PK] integer
usuario
character varying (100)
tabla_afectada
character varying (100)
operacion
character varying (20)
fecha
timestamp with time zone
datos_previos
jsonb



Unidad 6: Mantenimiento y Simulacro de Desastre

1. Informe Post-Incidente

- **Hora de inicio de la caída / Simulación:** 6:34 pm
- **Hora de detección / Notificación:** 6:35 pm
- **Hora de inicio del procedimiento de restauración:** 6:37 pm
- **Tiempo Total de Restauración (RTR):** 1 minuto y 15 segundos

2. Descripción del Escenario Simulado

Se simuló la corrupción intencional de datos en la tabla `pedidos` y el borrado accidental (Drop Schema) de la base de datos principal, dejando el sistema inoperativo.

3. Comandos Utilizados para la Recuperación

Para recuperar los datos, se generó un archivo de backup en formato SQL plano (plain). Para la restauración efectiva, se utilizó la herramienta **PSQL Tool** integrada dentro de pgAdmin. Mediante comandos directos de consola (`\i ruta/al/archivo.sql`), se ejecutó el script de backup, lo que permitió reintegrar los esquemas y las tablas evadiendo los errores de sintaxis del Query Tool convencional.

4. Resultado de la Verificación de Datos Post-Restauración

Se ejecutaron sentencias SELECT comprobando la existencia del esquema `public` y la integridad referencial de los datos de clientes y pedidos.

Query | Query History

```
1  SELECT table_name FROM information_schema.tables WHERE table_schema = 'public';
```

Data Output | Messages | Notifications

Showing rows: 1 to 8 | Page No: 1

	table_name name
1	auditoria_operaciones
2	detalle_pedidos
3	productos
4	metodos_pago
5	ventas_historicas_plan...
6	categorias
7	clientes
8	pedidos

Query | Query History

```
1  SELECT * FROM clientes LIMIT 5;
```

Data Output | Messages | Notifications

Showing rows: 1 to 5 | Page No: 1 of 1

	cliente_id [PK] integer	nombre character varying (100)	apellido character varying (100)	email character varying (150)	telefono character varying (20)	direccion text	fecha_regi timestamp
1	6	Lucas	Rodríguez	lucas.rodriguez@email.c...	+5491155555555	Mitre 654, La Plata	2026-01-2
2	7	Juan	Pérez	juan.perez@email.com	+5491155551111	Av. Corrientes 1234, CA...	2026-01-1
3	8	María	Gómez	maria.gomez@email.com	+5491155552222	Belgrano 456, Córdoba	2026-01-1
4	9	Ana	Martínez	ana.martinez@email.com	+5491155554444	Rivadavia 321, Rosario	2026-01-2
5	10	Carlos	López	carlos.lopez@email.com	+5491155553333	San Martín 789, Mendoza	2026-01-1

Query | Query History

```sql
SELECT
    p.pedido_id,
    c.nombre || ' ' || c.apellido AS cliente,
    p.fecha_pedido,
    p.estado,
    p.total
FROM pedidos p
JOIN clientes c ON p.cliente_id = c.cliente_id
LIMIT 5;
```

Data Output | Messages | Notifications

Showing rows: 1 to 5 | Page No: 1 of 1

	pedido_id [PK] integer	cliente text	fecha_pedido timestamp with time zone	estado estado_pedido_enum	total numeric (10,2)
1	7	Ana Martínez	2026-01-22 20:30:00-03	ENTREGADO	10600.00
2	8	Lucas Rodrígu...	2026-01-28 21:15:00-03	ENTREGADO	13300.00
3	9	Juan Pérez	2026-01-25 12:45:00-03	CANCELADO	5500.00
4	10	Juan Pérez	2026-01-12 13:20:00-03	ENTREGADO	10700.00
5	11	María Gómez	2026-01-15 21:00:00-03	ENTREGADO	14300.00

Unidad 7: Base de Datos NoSQL y Nuevas Formas de Almacenamiento

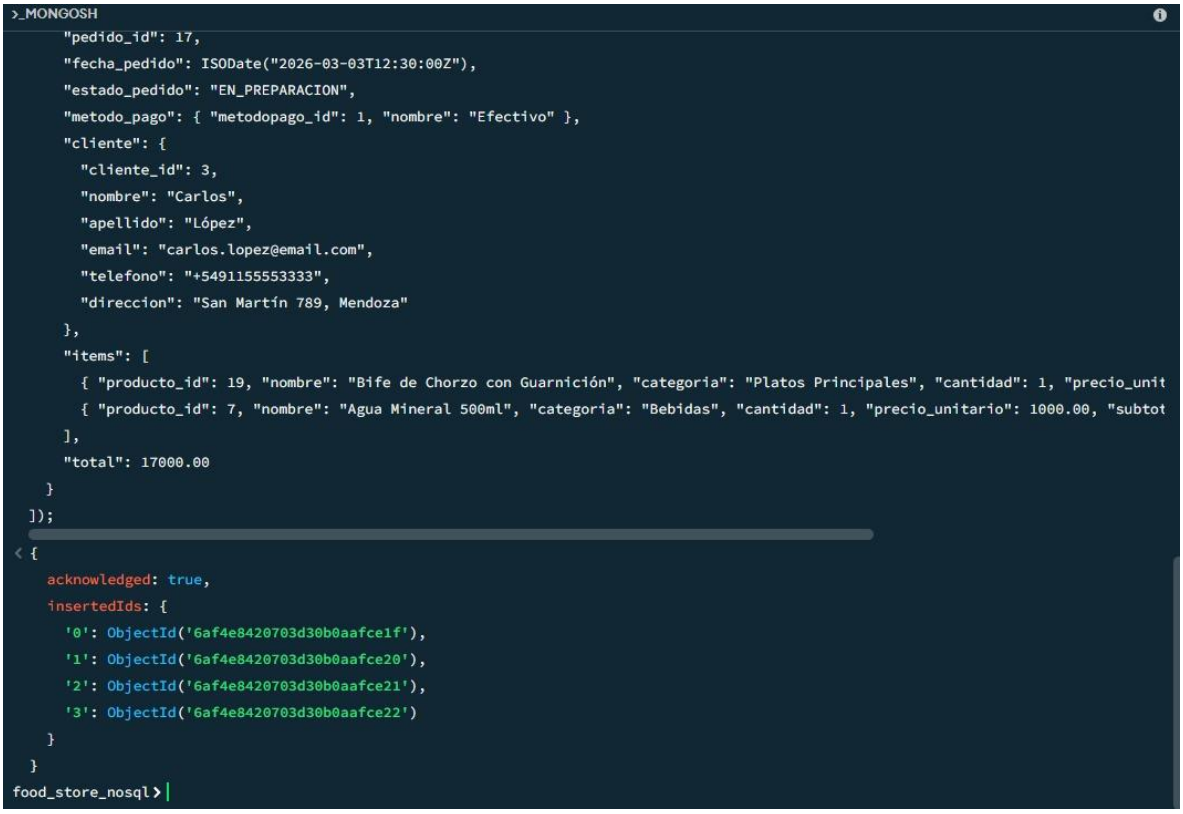
1. Comparación Técnica: NoSQL (MongoDB) vs Relacional (PostgreSQL)

Para un caso de uso centrado en el historial de pedidos, el modelo documental de MongoDB ofrece ventajas arquitectónicas notables frente al modelo relacional normalizado. En PostgreSQL, consultar una orden completa requiere ensamblar mediante operaciones JOIN la cabecera del pedido, los datos del cliente, el método de pago, el detalle de los productos y sus respectivas categorías a través de tablas separadas. Si bien esto garantiza una consistencia estricta y evita duplicidades (gracias a 3FN/BCNF), implica un costo de procesamiento en tiempo de lectura proporcional a la cantidad de uniones.

En contraste, el diseño documental de MongoDB aplica la estrategia de desnormalización por lectura (Embedding). Al incrustar al cliente y los ítems del detalle directamente dentro del documento del pedido, la orden se almacena como una unidad lógica autosuficiente en el disco. Cuando una aplicación de e-commerce solicita el detalle de una compra pasada, MongoDB recupera el documento completo mediante una única operación de lectura

indexada sin necesidad de realizar ningún JOIN. Esto reduce drásticamente la latencia y simplifica el mapeo de objetos en el backend.

No obstante, esta ventaja viene con contrapartidas. Si el cliente modifica su dirección, en el modelo documental esos datos quedan duplicados estáticamente en cada pedido histórico. Por esta razón, la arquitectura ideal suele ser híbrida: PostgreSQL maneja el inventario, usuarios y transacciones con integridad estricta, mientras que MongoDB brilla para persistir catálogos o generar el historial inmutable de compras.



```
> db.pedidos.find({ "cliente.email": "juan.perez@email.com" });
< {
    _id: ObjectId('6af4e8420703d30b0aafce1f'),
    pedido_id: 1,
    fecha_pedido: ISODate('2026-01-12T13:20:00.000Z'),
    estado_pedido: 'ENTREGADO',
    metodo_pago: {
      metodopago_id: 1,
      nombre: 'Efectivo'
    },
    cliente: {
      cliente_id: 1,
      nombre: 'Juan',
      apellido: 'Pérez',
      email: 'juan.perez@email.com',
      telefono: '+5491155551111',
      direccion: 'Av. Corrientes 1234, CABA'
    },
    items: [
      {
        producto_id: 1,
        nombre: 'Hamburguesa Clásica con Queso',
        categoria: 'Comida Rápida',
        cantidad: 1,
        precio_unitario: 8500,
        subtotal: 8500
      },
      {
        producto_id: 6,
        nombre: 'Coca-Cola 500ml',
        categoria: 'Bebidas',
        cantidad: 1,
        precio_unitario: 2200,
        subtotal: 2200
      }
    ],
    total: 10700
  }
```

```
> db.pedidos.aggregate([
    {
      $group: {
        _id: "$estado_pedido",
        total_recaudado: { $sum: "$total" },
        cantidad_pedidos: { $sum: 1 }
      }
    },
    {
      $sort: { total_recaudado: -1 }
    }
  ]);
< {
    _id: 'ENTREGADO',
    total_recaudado: 25000,
    cantidad_pedidos: 2
  }
  {
    _id: 'EN_PREPARACION',
    total_recaudado: 17000,
    cantidad_pedidos: 1
  }
  {
    _id: 'CANCELADO',
    total_recaudado: 5500,
    cantidad_pedidos: 1
  }
```

Unidad 8: Bases de Datos Móviles e Integración

1. Simulación de Conflicto de Sincronización Bidireccional

Imaginemos que el producto `producto_id = 1` ("Hamburguesa Clásica con Queso") tiene un stock de 50 unidades.

- **En el Servidor (PostgreSQL):** A las 14:00:00, se realiza una venta y el stock se actualiza a 48.
- **En la App Móvil (SQLite):** A las 14:05:00 (estando offline), el usuario administrador modifica el precio del mismo producto a $9000.00, pero el stock local sigue figurando desactualizado en 50.

Al intentar sincronizar los cambios hacia el servidor, se produce un conflicto de concurrencia y divergencia de estado sobre la misma tupla.

2. Estrategia de Resolución de Conflictos

Para resolver este conflicto de manera robusta, la estrategia estándar es **LWW (Last Write Wins - El último en escribir gana)** combinada con control de versión por marca temporal (`updated_at`). Al momento de la sincronización, el backend compara la marca temporal del dispositivo móvil frente al servidor. Gana la operación con la fecha más reciente (en este caso, se acepta el cambio de precio de las 14:05 de la app móvil). El inventario, por ser crítico, se puede manejar paralelamente mediante una estrategia "Server-Wins" o aplicando un cálculo de deltas, prevaleciendo siempre la consistencia financiera del servidor.

3. Code Review Final (Asistido por IA)

Hallazgo 1: Riesgo de Bloqueos Prolongados en `SELECT ... FOR UPDATE`

- **Análisis:** El uso de FOR UPDATE sobre la tabla productos es correcto para prevenir lecturas fantasma. Sin embargo, si la transacción no se mantiene corta, o si se bloquean múltiples productos en distinto orden, se expone el sistema a deadlocks (interbloqueos).
- **Recomendación:** Abarcar únicamente la sentencia crítica de lectura y el UPDATE inmediato, y ordenar siempre las consultas por `producto_id ASC` para mantener un orden determinista.
- **Decisión del equipo:** [Redactar aquí tu decisión: Ej. Aceptado, se implementará en la lógica del backend].

Hallazgo 2: Optimización del Trigger con Tablas de Transición

- **Análisis:** El trigger AFTER INSERT con `new_table` es excelente a nivel de rendimiento por sentencia. No obstante, si un pedido ingresa con productos duplicados en el mismo detalle, el `UPDATE ... FROM new_table` podría restar stock de forma errónea por la duplicidad de filas.
- **Recomendación:** Asegurar en la capa de inserción del backend que los ítems vengan consolidados y sumados por `producto_id` antes del INSERT.
- **Decisión del equipo:** [Redactar aquí tu decisión].

Hallazgo 3: Rendimiento de la Función de Ventana (`RANK() OVER`)

- **Análisis:** La consulta analítica para el ranking de clientes es óptima, pero al escalar a millones de registros, el motor realizará escaneos masivos en disco.
- **Recomendación:** Crear un Índice Parcial Compuesto en la tabla de pedidos que soporte el filtro y la agregación: `CREATE INDEX idx_pedidos_cliente_estado_total ON pedidos(cliente_id, estado) INCLUDE (total);`

- **Decisión del equipo:** El equipo decidio aceptar los 3 hallazgos y encontrar la forma de implementarlos en la versión final.

Bitácora DUIA (Uso de Inteligencia Artificial sobre datos)Herramienta utilizada: OpenCode (Gemini 3.0 Pro) en todas las fases.

- **Fase 1 (Unidad 4 - Migración):**
  - *Finalidad:* Generación de esquema heredado intencionalmente redundante y script de migración transaccional.
  - *Prompt principal empleado:* "Necesito cumplir con el requerimiento de migración de datos sin pérdida desde un esquema heredado. Genera el DDL para crear ventas_historicas_planas, un DML con 10 registros repetidos, y un script de migración SQL usando BEGIN y un plan de reversión ROLLBACK."
- **Fase 2 (Unidad 5 - Seguridad):**
  - *Finalidad:* Generación de esquema de roles (PoLP) y sistema de auditoría.
  - *Prompt principal empleado:* "Crea 3 roles (admin, backend, auditor). Aplica el principio de mínimo privilegio otorgando GRANT y REVOKE. Crea una tabla auditoria_operaciones y un trigger para registrar intentos de UPDATE o DELETE en pedidos."
- **Fase 3 (Unidad 6 - Mantenimiento):**
  - *Finalidad:* Creación del plan rutinario y comandos de restauración para simulacro.
  - *Prompt principal empleado:* "Escribe el script para automatizar VACUUM ANALYZE y REINDEX, y proporciona los comandos de consola exactos (pg_restore/psql) para hacer y restaurar un backup."
- **Fase 4 (Unidad 7 - NoSQL):**
  - *Finalidad:* Modelado documental en BSON, inserción y consultas.
  - *Prompt principal empleado:* "Diseña la estructura JSON para la colección pedidos incrustando los datos del cliente y un arreglo de ítems. Escribe el comando insertMany, una consulta find y una aggregate."
- **Fase 5 (Unidad 8 - Móvil y Code Review):**
  - *Finalidad:* Traducción DDL a SQLite y Code Review estricto.
  - *Prompt principal empleado:* "Traduce las tablas a SQLite ajustando tipos. Escribe la estrategia para resolver un conflicto de sincronización LWW. Actúa como Senior DBA y haz un Code Review con 3 hallazgos sobre mis scripts."